Final Project: Recommendation System with Implicit Feedback

Thien, Nguyen Van Minh
22127398
Faculty of Information Technology
University of Science, VNU-HCM
nvmthien22@clc.fitus.edu.vn

Ky, Dang Van
22127227
Faculty of Information Technology
University of Science, VNU-HCM
dvky22@clc.fitus.edu.vn

***Abstract*—**

***Index Terms*—Recommender Systems, Implicit Feedback**

I. Introduction

Recommendation systems play an important role in modern online platforms, including e-commerce, social media, video streaming, and news recommendation. In many real-world applications, user preferences are not expressed explicitly through ratings, but implicitly through behavioral signals such as clicks, views, purchases, likes, or watch time. This project focuses on recommendation systems with implicit feedback under the binary interaction setting, where observed interactions are treated as positive feedback and unobserved interactions are treated as missing or negative signals.

The task considered in this report is Top-K recommendation: given a user, the system ranks candidate items and returns the top K items that are most likely to match the user's preference. To study this problem, we review state-of-the-art approaches, implement and compare three selected models, and evaluate them on four benchmark datasets together with one dataset collected by our group.

A. Problem Statement

This project investigates how different state-of-the-art recommendation models perform in the implicit-feedback setting. In particular, we aim to answer the following questions:

- How should implicit feedback be represented and modeled?
- What are the strengths and limitations of different recommendation architectures on Top-K ranking tasks?
- How do dataset characteristics affect model performance?
- Which evaluation protocol and ranking metrics are appropriate for a fair comparison?

B. Objectives

The objectives of this report are as follows:

- Survey recent literature on recommendation systems with implicit feedback.
- Select three state-of-the-art models with available source code.
- Construct one self-collected dataset of comparable scale.
- Evaluate all selected models on five datasets in total.
- Analyze performance differences across models and datasets.

C. Research Scope

The scope of this project is limited to:

- Implicit feedback.
- Binary interaction representation.
- Top-K recommendation.
- Ranking-based evaluation metrics.
- A fixed evaluation protocol for fair comparison.

D. Contributions

The main contributions of our work are:

- A concise review of major research directions for implicit-feedback recommendation.
- An implementation and comparison of three representative SOTA methods.
- A new self-collected dataset with a documented collection pipeline.
- An experimental study over five datasets with multiple ranking metrics.
- An analysis of the influence of dataset properties on recommendation quality.

II. Related Work

This section reviews the evolution of recommendation systems for implicit feedback, summarizes major research branches, and motivates the selection of the three models used in our experiments.

A. Development Overview and Taxonomy

Recommendation methods for implicit feedback can be broadly organized into several families:

- Matrix factorization-based methods.
- Neighborhood-based methods.
- Neural recommendation methods.
- Graph-based recommendation methods.
- Self-supervised / contrastive learning methods.

For each family, briefly discuss:

- Core idea.
- Representative models.
- Main strengths.
- Main limitations.
- Historical significance and major breakthroughs.

B. Criteria for Selecting SOTA Models

We select our three models based on the following criteria:

- Strong performance reported in recent literature.

- Publicly available implementation.
- Clear methodological novelty.
- Representativeness of different research directions.
- Feasibility under our computational budget.

C. Chosen Model 1: LightGCN

a) *Core Idea:*

LightGCN is built on the intuition that the two most common components of Graph Convolutional Networks (GCNs) — feature transformation matrices and nonlinear activation functions — are actually redundant and harmful when applied to collaborative filtering with implicit feedback. Instead, the only meaningful operation is to let each user and item *aggregate* the embeddings of its neighbors on the user–item interaction graph. A user who interacted with many popular items will absorb those items' signals; an item interacted with by many active users will absorbs those users' signals. Stacking multiple such propagation layers captures higher-order connectivity (friends-of-friends effects), and the final representation is a weighted combination of all layer outputs.

b) *Architecture:*

LightGCN operates on a bipartite user–item graph $\mathcal{G} = (\mathcal{U} \cup \mathcal{I}, \mathcal{E})$, where an edge $(u, i) \in \mathcal{E}$ exists if user u has interacted with item i.

Layer-wise propagation. At layer k, each node aggregates its first-order neighbors without any learnable weight matrix or activation:

$$\boldsymbol{e}_u^{(k)} = \sum_{i \in \mathcal{N}_u} \frac{1}{\sqrt{|\mathcal{N}_u|}\sqrt{|\mathcal{N}_i|}} \boldsymbol{e}_i^{(k-1)}, \quad \boldsymbol{e}_i^{(k)} = \sum_{u \in \mathcal{N}_i} \frac{1}{\sqrt{|\mathcal{N}_i|}\sqrt{|\mathcal{N}_u|}} \boldsymbol{e}_u^{(k-1)}. \tag{1}$$

Layer combination. The final embedding is the mean of all $K+1$ layer outputs, with layer-0 being the trainable ID embedding:

$$\boldsymbol{e}_u = \sum_{k=0}^{K} \alpha_k \boldsymbol{e}_u^{(k)}, \quad \boldsymbol{e}_i = \sum_{k=0}^{K} \alpha_k \boldsymbol{e}_i^{(k)}, \tag{2}$$

where $\alpha_k = \frac{1}{K+1}$ by default.

Prediction and training. The predicted relevance score is the inner product $\hat{y}_{ui} = \boldsymbol{e}_u^\top \boldsymbol{e}_i$. The model is trained with the Bayesian Personalised Ranking (BPR) loss over observed positive and sampled negative pairs. The only trainable parameters are the $|\mathcal{U}| + |\mathcal{I}|$ initial ID embeddings of dimension d.

c) *Innovation:*

Prior graph-based recommenders such as NGCF inherited the full GCN recipe (weight matrices, ReLU activations) from node-classification settings. LightGCN empirically and theoretically demonstrates that these components do not help and often hurt performance in the recommendation context. By stripping the model down to its essential operation, LightGCN achieves higher accuracy with fewer parameters, faster training, and simpler hyper-parameter tuning. It also introduces the layer-combination trick, which enables multi-scale neighbourhood information without over-smoothing.

d) *Reason for Selection:*

LightGCN is the de-facto backbone for graph-based collaborative filtering and serves as a direct baseline against more advanced methods. It is widely reproduced, achieving strong results on Gowalla (Recall@20 = 0.1830, NDCG@20 = 0.1554), Yelp2018 (Recall@20 = 0.0649, NDCG@20 = 0.0530), and Amazon-book (Recall@20 = 0.0411, NDCG@20 = 0.0315). Its simplicity makes it easy to implement, audit, and extend, and its public PyTorch implementation ensures full reproducibility within our computational budget.

D. Chosen Model 2: SGL (Self-supervised Graph Learning)

a) *Core Idea:*

SGL addresses a fundamental weakness of LightGCN: the model learns only from *observed* positive interactions, leaving the vast majority of the interaction space unexplored. Users with few interactions (long-tail users) receive poor representations because there is simply not enough supervision signal. SGL introduces a self-supervised auxiliary task on top of the main recommendation objective. By creating two randomly *augmented views* of the user–item graph and training the model to produce consistent representations of the same node across views, SGL derives additional supervision from the graph structure itself — without requiring any extra labels.

b) *Architecture:*

SGL wraps LightGCN with a graph-augmentation and contrastive-learning module.

Graph augmentation. Three stochastic operators produce an augmented graph $\tilde{\mathcal{G}}$ from the original $\mathcal{G}$:

- **Node Dropout (ND)**: randomly mask a fraction ρ of users or items and all their edges.
- **Edge Dropout (ED)**: randomly remove a fraction ρ of edges.
- **Random Walk (RW)**: for each node, sample a fixed-length random walk to define its local neighbourhood.

Two independent augmented views $\tilde{\mathcal{G}}'$ and $\tilde{\mathcal{G}}''$ are generated at each training step.

Contrastive objective. LightGCN is applied to both views to obtain embeddings $\boldsymbol{z}_{u'}$ and $\boldsymbol{z}_{u''}$ for each user u (and analogously for items). The InfoNCE contrastive loss maximises agreement between the two views of the same node while repelling all other nodes:

$$\mathcal{L}_{\text{ssl}} = \sum_{u \in \mathcal{U}} -\log \frac{\exp\left(\text{sim}\, \frac{\boldsymbol{z}_{u'}, \boldsymbol{z}_{u''}}{\tau}\right)}{\sum_{v \in \mathcal{U}} \exp\left(\text{sim}\, \frac{\boldsymbol{z}_{u'}, \boldsymbol{z}_{v''}}{\tau}\right)}, \tag{3}$$

where $\text{sim}(\cdot, \cdot)$ is cosine similarity and τ is a temperature hyper-parameter. A symmetric term over items is added.

Joint training. The final objective is:

$$\mathcal{L} = \mathcal{L}_{\text{BPR}} + \lambda \mathcal{L}_{\text{ssl}}, \tag{4}$$

where λ balances the two tasks. The backbone weights are shared across views so contrastive training directly regularises the main recommendation embeddings.

c) *Innovation:*

SGL is the first work to systematically apply self-supervised contrastive learning to graph-based collaborative filtering. The key insight is that graph augmentation creates diverse yet semantically consistent views, and aligning them encourages the encoder to learn representations that are robust to structural noise and missing data. This directly addresses the long-tail and sparsity problems that limit LightGCN. Compared to data augmentation methods that require item content or social links, SGL operates purely on the interaction graph, making it applicable to any standard implicit-feedback dataset.

d) *Reason for Selection:*

SGL consistently outperforms LightGCN and all pre-2021 CF baselines on standard benchmarks, with reported gains of roughly 10–15% in Recall@20: Gowalla (0.2263 / 0.1917), Yelp2018 (0.0815 / 0.0659), Amazon-book (0.0712 / 0.0556). It represents the first successful integration of SSL into the GNN-based CF paradigm, making it a key model for understanding how auxiliary signals improve recommendation quality. Its public implementation and compatibility with LightGCN allow direct ablation experiments within the same framework.

E. Chosen Model 3: SimGCL (Simple Graph Contrastive Learning)

a) *Core Idea:*

SimGCL revisits SGL and asks: *is graph augmentation actually the source of the performance gains, or is it something else?* Through theoretical analysis of the *uniformity–alignment* decomposition of the InfoNCE loss, the authors show that what truly matters is the *uniformity* pressure that contrastive learning exerts on the embedding space — not the specific graph perturbation strategy. Based on this finding, SimGCL replaces the expensive graph-augmentation module with the simplest possible operation: adding small uniform random noise directly to the node embeddings to generate contrastive views. The result is a model that is simultaneously simpler, faster, and more accurate than SGL.

b) *Architecture:*

SimGCL keeps the LightGCN encoder intact and changes only how contrastive views are produced.

Noise-based view generation. Given the final LightGCN embedding $\boldsymbol{e}_u$ of user u, two perturbed views are created by adding independent random noise vectors:

$$\boldsymbol{z}_{u'} = \boldsymbol{e}_u + \boldsymbol{\Delta}_{u'}, \quad \boldsymbol{z}_{u''} = \boldsymbol{e}_u + \boldsymbol{\Delta}_{u''}, \tag{5}$$

where each element of $\boldsymbol{\Delta}$ is sampled uniformly from $[-\varepsilon, \varepsilon]$ and then ℓ_2-normalised. The single hyper-parameter ε controls the magnitude of perturbation.

Contrastive and joint loss. The InfoNCE loss is identical to SGL:

$$\mathcal{L}_{\text{cl}} = \sum_{u \in \mathcal{U}} -\log \frac{\exp\left(\text{sim}\, \frac{\boldsymbol{z}_{u'}, \boldsymbol{z}_{u''}}{\tau}\right)}{\sum_{v \in \mathcal{U}} \exp\left(\text{sim}\, \frac{\boldsymbol{z}_{u'}, \boldsymbol{z}_{v''}}{\tau}\right)}, \tag{6}$$

and the total objective is $\mathcal{L} = \mathcal{L}_{\text{BPR}} + \lambda \mathcal{L}_{\text{cl}}$. Because no graph manipulation is needed, SimGCL requires only a single forward pass through LightGCN per training step, matching the computational cost of vanilla LightGCN while providing the regularisation benefits of contrastive learning.

c) *Innovation:*

SimGCL makes two fundamental contributions. First, it provides a theoretical explanation for *why* SSL improves CF: contrastive learning imposes uniformity on the embedding distribution, reducing popularity bias and improving coverage of the item space. Second, it demonstrates that complex, hand-crafted graph augmentation strategies are unnecessary — simple additive noise achieves equal or better uniformity–alignment trade-offs. This finding simplifies the design space for SSL-based recommenders and opens the door to even lighter future methods.

d) *Reason for Selection:*

SimGCL achieves state-of-the-art results among general CF methods on all three standard benchmarks: Gowalla (Recall@20 = 0.2313, NDCG@20 = 0.1962), Yelp2018 (Recall@20 = 0.0894, NDCG@20 = 0.0735), and Amazon-book (Recall@20 = 0.0834, NDCG@20 = 0.0671), outperforming SGL on every metric. As the most recent and theoretically grounded model in our trio, SimGCL provides a meaningful upper-bound reference point. Its training speed (comparable to LightGCN) is particularly advantageous given our computational constraints, and the SELFRec codebase offers a unified, extensible implementation of all three models.

F. Comparison of the Three Approaches

III. Datasets, Evaluation Metrics, and Implementation

This section describes the five datasets, preprocessing procedures, evaluation metrics, and implementation details.

A. Overview of the Five Datasets

We conduct experiments on five datasets in total:

- One self-collected dataset.
- Four benchmark datasets.

B. Self-Collected Dataset

a) *Collection Objective:*

[State the target scale: around 1M interactions, expected number of users/items, and why this source is suitable.]

b) *Data Source:*

[Describe the platform/source, access method, API or crawling constraints, and why it was selected.]

c) *Collection Pipeline:*

[Replace the placeholder above with an actual diagram.]

d) *Data Schema:*

e) *Preprocessing:*

[Describe how duplicates, missing values, invalid records, timestamps, user/item IDs, and interaction filtering were handled.]

f) *Statistics and Characteristics:*

g) *Difficulties and Solutions:*

[Describe practical difficulties such as access limits, noisy records, imbalance, missing fields, and how your group addressed them.]

C. Benchmark Dataset 1

a) *Description:* [Describe the dataset domain and its characteristics.]

b) *Statistics:* [Provide key statistics.]

D. Benchmark Dataset 2

a) *Description:* [Describe the dataset domain and its characteristics.]

b) *Statistics:* [Provide key statistics.]

E. Benchmark Dataset 3

a) *Description:* [Describe the dataset domain and its characteristics.]

b) *Statistics:* [Provide key statistics.]

TABLE I: Comparison among the three selected SOTA methods.

Model	Main Idea	Strengths	Weaknesses	Complexity / Notes
LightGCN (He et al., SIGIR 2020)	Simplifies GCN by removing feature transformation and nonlinear activation; uses only weighted neighborhood aggregation over the user–item graph.	Lightweight and fast; strong baseline; highly reproducible; linear complexity in graph edges.	No auxiliary self-supervised signal; sensitive to data sparsity; cannot model sequential user behavior.	$O(L \cdot \lvert\mathcal{E}\rvert \cdot d)$; BPR loss; 3–4 propagation layers typical.
SGL (Wu et al., SIGIR 2021)	Augments the user–item graph via node dropout, edge dropout, or random walk to create two contrastive views; optimizes InfoNCE loss alongside BPR.	Significantly improves over LightGCN on sparse data; robust to noisy and long-tail interactions; boosts uniformity of embeddings.	Graph augmentation triples forward-pass cost; sensitive to augmentation ratio and temperature τ; hyperparameter-heavy.	$O(3L \cdot \lvert\mathcal{E}\rvert \cdot d)$; BPR + InfoNCE; requires tuning τ, drop ratio, and SSL weight λ.
SimGCL (Yu et al., SIGIR 2022)	Replaces graph augmentation with uniform noise perturbation directly on embeddings to generate contrastive views; shows graph structure modification is unnecessary.	State-of-the-art accuracy; training speed comparable to LightGCN; theoretically grounded (uniformity–alignment analysis); simple to implement.	Focused on general CF; does not model item sequences; noise magnitude ε requires careful tuning per dataset.	$O(L \cdot \lvert\mathcal{E}\rvert \cdot d)$; BPR + InfoNCE; single extra hyperparameter ε for noise magnitude.

F. Benchmark Dataset 4

a) *Description:* [Describe the dataset domain and its characteristics.]

b) *Statistics:* [Provide key statistics.]

G. Unified Preprocessing Strategy

[Explain the common preprocessing applied consistently across all datasets, such as binarization, filtering by minimum interactions, train-validation-test split, and ID remapping.]

TABLE II: Overview of the five datasets used in this project.

Dataset	Domain	#Users	#Items	#Interactions	Sparsity / Notes
Collected Dataset	TODO	TODO	TODO	TODO	TODO
Benchmark 1	TODO	TODO	TODO	TODO	TODO
Benchmark 2	TODO	TODO	TODO	TODO	TODO
Benchmark 3	TODO	TODO	TODO	TODO	TODO
Benchmark 4	TODO	TODO	TODO	TODO	TODO

Fig. 1: Data collection and preprocessing pipeline for the self-collected dataset.

TABLE III: Schema of the self-collected dataset.

Field	Type	Description
user_id	string/int	Unique user identifier
item_id	string/int	Unique item identifier
timestamp	datetime/int	Interaction timestamp
event_type	string	Type of user interaction
context	optional	Additional metadata if available

TABLE IV: Statistics of the self-collected dataset.

Statistic	**Value**
Users	TODO
Items	TODO
Interactions	TODO
Average interactions per user	TODO
Average interactions per item	TODO
Sparsity	TODO

H. Evaluation Metrics

We evaluate recommendation quality using ranking metrics. These metrics can be roughly grouped into two categories.

a) *Accuracy-Oriented Metrics:*

- Precision@K.
- Recall@K.
- Hit Rate@K.

[Explain each metric briefly, including what it measures and when it is useful.]

Fig. 2: Architecture of Model 1.

b) *Order-Aware Metrics:*

- MAP@K.
- MRR@K.
- NDCG@K.

[Explain each metric briefly, especially why ranking order matters.]

c) *Additional Metrics (Optional):*

[Discuss whether other metrics are used, such as AUC, coverage, diversity, novelty, or popularity bias related indicators.]

I. Evaluation Protocol

[Clearly define the "rules of the game" for evaluation.]

- Train/validation/test split strategy.
- Leave-one-out or ratio-based split.
- Negative sampling or full ranking.
- Candidate set construction.
- How Top-K metrics are computed.
- Fairness and reproducibility considerations.

J. Implementation Details

[Describe the implementation setup.]

- Programming language and framework.
- Hardware resources.
- Hyperparameter search strategy.
- Training epochs and early stopping.
- Random seeds.
- Number of repeated runs per experiment.

IV. Experimental Methods

A. Experimental Design

We evaluate three models on five datasets, resulting in fifteen experimental settings. For each setting, we run the experiment multiple times and report the average result to reduce randomness.

[Briefly describe the design rationale and fairness considerations.]

B. Model 1: [Name]

a) *Core Idea:* [Summarize the method.]

b) *Architecture:* [Explain the modules and data flow.]

c) *Innovation:* [Highlight the novelty.]

Fig. 3: Architecture of Model 2.

Fig. 4: Architecture of Model 3.

C. Model 2: [Name]

a) *Core Idea:* [Summarize the method.]

b) *Architecture:* [Explain the modules and data flow.]

c) *Innovation:* [Highlight the novelty.]

D. Model 3: [Name]

a) *Core Idea:* [Summarize the method.]

b) *Architecture:* [Explain the modules and data flow.]

c) *Innovation:* [Highlight the novelty.]

E. Summary of the Three Methods

[Summarize the conceptual differences and expected behavior of the three methods before presenting results.]

V. Experiments and Results

A. Training Notes

For each experiment, we record:

- Training time.
- Hardware resources.
- Convergence behavior.
- Stability across repeated runs.

B. Main Results

C. Comparison Across Datasets

[Analyze how each model behaves on different datasets.]

TABLE V: Main experimental results on five datasets.

Dataset	Model	Recall@20	NDCG@20	HR@20	MRR@20
Dataset 1	Model 1	TODO	TODO	TODO	TODO
Dataset 1	Model 2	TODO	TODO	TODO	TODO
Dataset 1	Model 3	TODO	TODO	TODO	TODO
Dataset 2	Model 1	TODO	TODO	TODO	TODO
Dataset 2	Model 2	TODO	TODO	TODO	TODO
Dataset 2	Model 3	TODO	TODO	TODO	TODO
Dataset 3	Model 1	TODO	TODO	TODO	TODO
Dataset 3	Model 2	TODO	TODO	TODO	TODO
Dataset 3	Model 3	TODO	TODO	TODO	TODO
Dataset 4	Model 1	TODO	TODO	TODO	TODO
Dataset 4	Model 2	TODO	TODO	TODO	TODO
Dataset 4	Model 3	TODO	TODO	TODO	TODO
Dataset 5	Model 1	TODO	TODO	TODO	TODO
Dataset 5	Model 2	TODO	TODO	TODO	TODO
Dataset 5	Model 3	TODO	TODO	TODO	TODO

D. Impact of Dataset Characteristics

[Discuss sparsity, scale, popularity skew, domain differences, or density.]

E. Additional Analysis

Possible additional analyses include:

- Ablation study.
- Sensitivity to K.
- Sensitivity to hyperparameters.
- Training efficiency comparison.
- Case study / recommendation examples.

F. Discussion

[Summarize the key findings and practical implications.]

VI. Conclusion

This report studied recommendation systems with implicit feedback in the binary Top-K recommendation setting. We reviewed three state-of-the-art methods, collected one dataset, and benchmarked all methods on five datasets.

The major findings of this study can be summarized as follows:

- [Finding 1]
- [Finding 2]
- [Finding 3]

The strengths and weaknesses of the selected models were identified through both quantitative results and qualitative analysis. In addition, we found that dataset characteristics have a significant effect on ranking performance.

Future work may include:

- Testing more advanced self-supervised or graph-based models.
- Improving the quality and diversity of the collected dataset.
- Exploring additional evaluation metrics such as diversity and novelty.
- Extending the study to sequential or session-based recommendation.

References

VII. Appendix

A. A. Hyperparameter Search Space

[Optional]

B. B. Additional Experimental Results

[Optional]

C. C. Sample Cases from the Collected Dataset

[Optional]